This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

PeLK: Parameter-efficient Large Kernel ConvNets with Peripheral Convolution

Honghao Chen[1,2*] Xiangxiang Chu[3] Yongjian Ren[1,2] Xin Zhao[1,2] Kaiqi Huang[1,2,4†]

[1]Institute of Automation, Chinese Academy of Sciences
[2]School of Artificial Intelligence, University of Chinese Academy of Sciences
[3] Meituan [4] CAS Center for Excellence in Brain Science and Intelligence Technology

Abstract

Recently, some large kernel convnets strike back with appealing performance and efficiency. However, given the square complexity of convolution, scaling up kernels can bring about an enormous amount of parameters and the proliferated parameters can induce severe optimization problem. Due to these issues, current CNNs compromise to scale up to 51×51 *in the form of stripe convolution (i.e.,* $51 \times 5 + 5 \times 51$*) and start to saturate as the kernel size continues growing. In this paper, we delve into addressing these vital issues and explore whether we can continue scaling up kernels for more performance gains. Inspired by human vision, we propose a human-like peripheral convolution that efficiently reduces over 90% parameter count of dense grid convolution through parameter sharing, and manage to scale up kernel size to extremely large. Our peripheral convolution behaves highly similar to human, reducing the complexity of convolution from* $O(K^2)$ *to* $O(\log K)$ *without backfiring performance. Built on this, we propose Parameter-efficient Large Kernel Network (**PeLK**). Our PeLK outperforms modern vision Transformers and ConvNet architectures like Swin, ConvNeXt, RepLKNet and SLaK on various vision tasks including ImageNet classification, semantic segmentation on ADE20K and object detection on MS COCO. For the first time, we successfully scale up the kernel size of CNNs to an unprecedented* 101×101 *and demonstrate consistent improvements.*

1. Introduction

Convolutional Neural Networks (CNNs) have played a pivotal role in machine learning for decades [16, 19, 20, 35]. However, their dominance has been greatly challenged by Vision Transformers (ViTs) [6, 12, 24, 42, 47] over recent years. Some works [32, 44] attribute the powerful performance of ViTs to their large receptive fields: Facilitated by self-attention mechanism, ViTs can capture context information from a large spatial scope and model long-range dependencies. Inspired by this, recent advances in CNNs [11, 23, 25] have revealed that when equipped with large kernel size (e.g., 31×31), pure CNN architecture can perform on par with or even better than state-of-the-art ViTs on various vision tasks.

Although large kernel convnets exhibit strong performance and appealing efficiency, a fatal problem exists: the square complexity $O(K^2)$ with respect to kernel size K. Due to this problem, directly scaling up kernels will bring about a huge amount of parameters. For instance, the parameter of a 31×31 kernel is more than $100\times$ larger than that of a typical 3×3 counterpart in ResNet [16] and about $20\times$ as many as that of the 7×7 kernel used in ConvNeXt [25]. The proliferated parameters subsequently induce severe optimization problem, making it useless or even harmful to directly scale up kernel size [11, 23, 25]. To solve, RepLKNet [11] re-parameterize a 5×5 kernel parallel to the large one to make up the optimization issue, SLaK [23] compromise to use stripe convolution to reduce the complexity to linear and scales up to 51×51 (i.e., $51 \times 5 + 5 \times 51$). However, this is still a limited interaction range for the resolution of downstream tasks (e.g., 2048×512 on ADE20K) and more importantly, stripe convolution lacks the range perception of dense convolution, thus we conjecture it may undermine the model's spatial perception capacity.

In this paper, we first conduct a comprehensive dissection of convolution forms under a unified modern framework (i.e., SLaK [23]). We empirically verify our conjecture that dense grid convolution outperforms stripe convolution with consistent improvements across multiple kernel sizes. This phenomenon holds not only for classification task, but even more pronounced for downstream tasks, indicating the essential advantage of dense convolution over stripe form. Nevertheless, as mentioned above, the square complexity of large dense convolution leads to the proliferated **parameters**, causing rapidly increasing model size, greater optimization difficulty and thus preventing it from

*Work done during internship at Meituan Inc.
†Corresponding author.

further scaling. This non-trivial problem naturally leads to a question: Is there a way to preserve the form of dense grid convolution while reducing the parameters required? And if so, can we further scale up dense grid convolution for more performance gains?

Unlike the dense computation of convolution or self-attention, human vision possesses a more efficient visual processing mechanism termed peripheral vision [21]. Specifically, human vision partitions the entire visual field into central region and peripheral region conditioned on the distance to the center of the gaze, and the number of photoreceptor cells (cones and rods) in the central region is more than 100 times that in the peripheral region [36]. Such a physiological structure gives human vision the characteristic of blur perception: we have strong perception and see clearly in the central region, recognizing shapes and colors; whereas in the peripheral region, the visual field is blurred and the resolution decreases so we can only recognize abstract visual features such as motion and high-level contexts. This mechanism enables us to perceive important details within a small portion of the visual field ($< 5\%$) while minimizing unnecessary information in the remaining portion ($> 95\%$), thereby facilitating efficient visual processing in the human brain [2, 9, 10, 26, 33, 34, 48, 50].

Inspired by human vision and to answer the question above, we propose a novel peripheral convolution to reduce the parameter complexity of convolutions from $O(K^2)$ to $O(\log K)$ while maintaining the dense computational form. Our peripheral convolution consists of three designs: **i)** Focus and blur mechanism. We keep fine-grained parameters in the central region of the convolution kernel and use wide-range parameter sharing in the peripheral regions; **ii)** Exponentially-increasing sharing granularity. Our sharing grid grows in an exponentially-increasing way, which is more effective than fixed granularity; **iii)** Kernel-wise positional embedding. We introduce kernel-wise positional embedding to solve the problem of detail blurring caused by wide-range peripheral sharing in an elegant and cheap way. Since our peripheral convolution dramatically reduces the parameters for large kernels (over 90%), we are able to design large dense kernel convnets with strong performance.

Built upon the peripheral convolution above, we propose Parameter-efficient Large Kernel Network (**PeLK**), a new pure CNN architecture with Effective Receptive Field (ERF) growing exponentially with parameters. Facilitated by the elaborately designed parameter sharing mechanism, PeLK scales up kernel size at a remarkably minor parameter cost, realizing extremely large dense kernel (e.g., 51×51, 101×101) with consistent improvements. Our PeLK achieves state-of-the-art performance across a variety of vision tasks, exhibiting the potential of pure CNN architecture when equipped with extremely large kernel size.

PeLK is shown to be able to cover a much larger ERF region than prior large kernel paradigms, which we believe leads to its strong performance. More interestingly, our analysis and ablations demonstrate that the optimal design principles of peripheral convolution share striking similarities with human vision, suggesting that biologically inspired mechanisms can be promising candidates for designing strong modern networks.

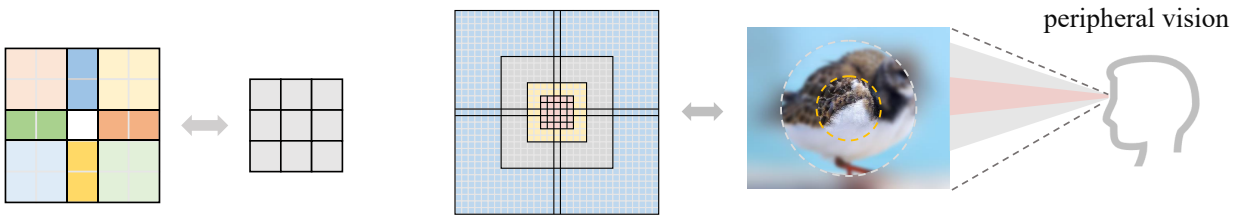
2. Related Work

2.1. Large Kernel Convolutional Networks

Large kernel convolutional networks can date back to a few old fashion models from the early days of deep learning [19, 38, 39]. After VGG-Net [35], it becomes a common practice to use a stack of small kernels (e.g., 1×1 or 3×3) to obtain a large receptive field over the past decade. Global Convolutional Network (GCNs) [30] enlarges the kernel size to 15 by employing a combination of stripe convolutions ($1 \times \mathrm{M} + \mathrm{M} \times 1$) to improve the semantic segmentation task. However, the proposed method is reported to harm the performance on ImageNet. Recently, large kernel convnets strike back with appealing performance [11, 23, 25, 43]. ConvMixer [43] use 9×9 depthwise convolution to replace the spatial mixer of ViT [12] and MLP-Mixer [40] (i.e., self-attention block and fully-connection block respectively). ConvNeXt [25] aligns with Swin's [24] design philosophy to explore a strong modern CNN architecture equipped with 7×7 depthwise convolution. RepLKNet [11] impressively scales up the kernel size to 31×31 by re-parameterizing a small kernel (e.g., 5×5) parallel to it and performs on par with Swin Transformer [24]. Our work is also inspired by LargeKernel3D [5], which introduces large kernel design into 3D networks and scales up to $17 \times 17 \times 17$. In contrast, we explore the extremety of 2D universal convolution, scaling up to a much larger 101×101 in a human-like pattern. SLaK [23] combines decomposed convolution with dynamic sparsity to scale up kernels to 51×51 in the form of stripe convolution (e.g., $51 \times 5 + 5 \times 51$). However, it starts to saturate as the kernel size continuous growing. Different from those prior arts, we investigate which kind of convolution form is more effective in large kernel designs. More importantly, we explore the design of extremely large dense kernel and test whether it can bring further gains.

2.2. Peripheral Vision for Machine Learning

Human vision has a special visual processing system termed peripheral vision [21]. It partitions the entire visual field into multiple contour regions depending on the distances to the fovea, each characterized by a distinct resolution granularity for recognition. The work of Rosenholtz [33] discusses in depth important findings and existing myths about peripheral vision, suggesting that peripheral vision is more crucial to human perception on a range of differ-



(a) Parameter Sharing. (b) Peripheral Convolution.

Figure 1. **(a) Illustration of parameter sharing.** Using a 3×3 convolution to parameterize a 5×5 convolution, the positions with the same color share the same parameter. The corresponding sharing grid is $[2, 1, 2]$. **(b) Illustration of peripheral convolution.** Our sharing grid contains two designs: i) focus and blur mechanism; ii) exponentially-increasing sharing grid.

ent tasks than previously thought. Following this, many studies [2, 9, 10, 26, 34, 50] have been devoted to uncovering the underlying principles and deep implications of peripheral vision mechanisms. Since peripheral vision plays such a vital role in human vision, a number of pioneering works [10, 13–15, 27, 46] dig into the linkage between peripheral vision and machine vision (e.g., CNNs). [45] introduces a biologically-inspired mechanism to improve the robustness of neural networks to small adversarial perturbations. FoveaTer [18] uses radial-polar pooling regions to dynamically allocate more fixation/computational resources to more challenging images. PerViT [29] proposes to incorporate peripheral position encoding to the multi-head self-attention layers to partition the visual field into diverse peripheral regions, showing that the network learns to perceive visual data similarly to the way that human vision does. Continuing previous study, this paper explores to blending human peripheral vision with large kernel convnets, and introduces a novel peripheral convolution to efficiently reduce dense convolution's parameters.

3. Dense Outperforms Stripe Consistently

We first investigate whether dense grid convolutions are better than stripe convolutions. We take a unified modern framework SLaK [23] to conduct this study. According to RepLKNet [11], large kernel convolution boosts downstream tasks much more than ImageNet classification. So we not only evaluate on ImageNet-1K but also on ADE20K as our benchmark. We adopt the efficient large-kernel implementation developed by MegEngine [1] in this paper.

Following SLaK [23], we train all models for a 120-epoch schedule on ImageNet. The data augmentations, regularization and hyper-parameters are all set the same. We then use the pretrained models as the backbones on ADE20K. Specifically, we use the UperNet [52] implemented by MMSegmentation [7] with the 80K-iteration training schedule. We do not use any advanced techniques nor custom algorithms since we seek to evaluate the backbone only.

SLaK introduce a two-step recipe for scaling up kernel to 51×51: 1) Decomposing a large kernel into two rectangular, parallel kernels; 2) Using dynamic sparsity and expanding more width. In order to thoroughly analyze the effect of convolution form, we conduct experiments both w/ and w/o sparsity. By default, we re-parameterize a 5×5 convolution to ease the optimization problem as taken by SLaK and RepLKNet. The results of Table 1 show that dense grid convolution exceeds stripe convolution regardless of dynamic sparsity.

We further explore convolution forms (i.e., K×K v.s. K×N) under different kernel sizes. Specifically, we fix the shorter edge of SLaK's stripe conv to be 5 as the default setting (N=5), and then gradually decrease K from 51 to 7. We do not use dynamic sparsity to give a sheer ablation on convolutional forms. As shown in Fig. 2, dense grid convolution outperforms stripe convolution consistently among multiple kernel sizes and the gains increase with the kernel size, demonstrating the essential advantage of dense grid large kernel convolution.

Nevertheless, as discussed in Section 1, the square complexity of dense grid convolution can bring about proliferated parameters. For instance, as shown in Fig. 2, scaling up kernel from 7 to 51 only bring about 7.3× params for stripe conv while that for dense conv is 53.1×. Given that the human's peripheral vision has only a minimal number of photoreceptor cells in the peripheral regions, we argue that dense parameters are not necessary for peripheral interactions. Motivated by this, we seek to reduce parameter complexity by introducing the peripheral vision mechanism while preserving the dense computation to keep dense con-

Table 1. Comparison w/ and w/o dynamic sparsity. Dense convolution outperforms stripe convolution both on ImageNet and ADE20K.

Method	Kernel	Spasity	Acc	mIoU
SLaK-51	51×5 + 5×51	w/	81.6	46.5
RepLK-51	51×51	w/	81.7	46.9 (+0.4)
SLaK-51	51×5 + 5×51	w/o	81.3	46.1
RepLK-51	51×51	w/o	81.6	46.6 (+0.5)

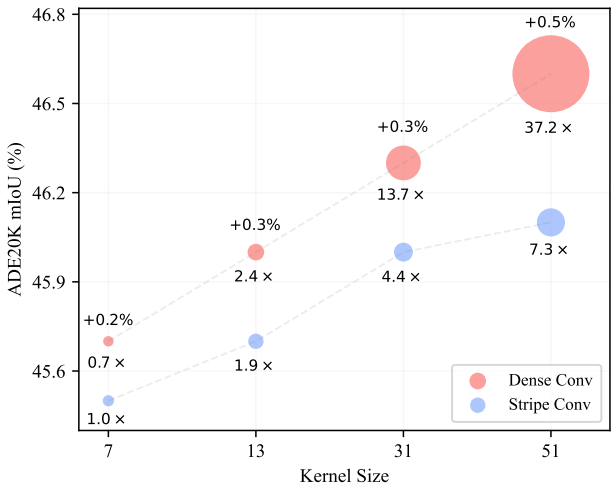


Figure 2. **Comparison under different kernel sizes.** We depict the mIoU gains on ADE20K and the multiple of convolutional parameters. Dense grid convolution exceeds stripe convolution consistently but brings rapidly-increasing parameters.

volution's strong performance.

4. Parameter-efficient Large Kernel Network

4.1. Peripheral Convolution

Formally, a standard 2D convolution kernel consists of a 4-D vector: $\mathrm{w} \in \mathbb{R}^{c_{\mathrm{in}} \times c_{\mathrm{out}} \times \mathrm{k} \times \mathrm{k}}$, where c_{in} stands for input channels, c_{out} is output channels, and k means the spatial kernel dimension. We seek to parameterize w by a smaller kernel $\mathrm{w}_\theta \in \mathbb{R}^{c_{\mathrm{in}} \times c_{\mathrm{out}} \times \mathrm{k}' \times \mathrm{k}'}$ through spatial-wise parameter sharing, where $0 < \mathrm{k}' \leq \mathrm{k}$.

Firstly, we define the sharing grid $S = [s_0, s_1, ..., s_{\mathrm{k}'-1}]$, where $\sum_{i=0}^{\mathrm{k}'-1} s_i = \mathrm{k}$. According to S, we partition the $\mathrm{k} \times \mathrm{k}$ positions into $\mathrm{k}' \times \mathrm{k}'$ regions:

for $a, b = 0, 1, ..., \mathrm{k}' - 1$,

$$Z_{a,b} = \left\{ (x, y) \middle| \sum_{i=0}^{a-1} s_i \leq x < \sum_{i=0}^{a} s_i, \sum_{j=0}^{b-1} s_j \leq y < \sum_{j=0}^{b} s_j \right\} \tag{1}$$

For brevity, we stipulate that $\sum_{i=0}^{-1} s_i = 0$ in Eq. 1. Then for any position $(x, y) \in Z_{a,b}$, we set $\mathrm{w}(x, y) = \mathrm{w}_\theta(a, b)$. In this way, we can utilize a small kernel to parameterize a much larger kernel, achieving spatial-wise parameter sharing. Fig. 1a depicts the illustration of this design.

Next, we elaborate on the key designs of our peripheral convolution. We denote the kernel radius of w_θ as r. For easier comprehension, here we reformulate the sharing grid into an axisymmetric form: $S = [\bar{s}_{-r}, \bar{s}_{-r+1}, ..., \bar{s}_{-1}, \bar{s}_0, \bar{s}_1, ..., \bar{s}_{r-1}, \bar{s}_r]$, where $r = \frac{\mathrm{k}'-1}{2}$.

Akin to human's peripheral vision, the sharing grid of our peripheral convolution mainly consists of two core designs: **i) Focus and blur mechanism.** As shown in Fig. 1b,

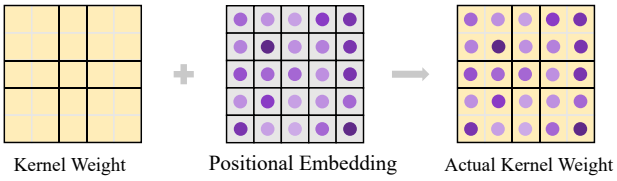


Figure 3. **Illustration of kernel-wise positional embedding.** The position embedding enables the kernel to distinguish specific positions in the sharing region, making up the detail-capturing ability of large kernels.

We keep fine-grained parameters in the central region of the convolution kernel, where the sharing grid is set to 1 (i.e., not sharing). For the peripheral region, we utilize large-range parameter sharing to exploit the spatial redundancy of peripheral vision. We demonstrate in Section 5.4 that the fine granularity in the central region is of vital importance, while the peripheral region can withstand a wide range of parameter sharing without backfiring performance; **ii) Exponentially-increasing sharing granularity.** Human vision declines in a quasi-exponential mode [31]. Inspired by this, we design our sharing grid to grow in an exponentially-increasing way. This design can elegantly reduce the parameter complexity of convolution from $O(K^2)$ to $O(\log K)$, making it possible to further enlarge dense convolution's kernel size. Specifically, the sharing grid S is constructed by:

$$\bar{s}_i = \begin{cases} 1, & \text{if } |i| \leq r_c \\ \mathrm{m}^{(|i| - r_c)}, & \text{if } r_c < |i| \leq r \end{cases} \tag{2}$$

where r_c is the radius of the central fine-grained region, m is the base of the exponential growth and m is set to 2 by default.

4.2. Kernel-wise Positional Embedding

Despite that the proposed peripheral convolution effectively reduces the parameters for dense convolution, the large range of parameter sharing may bring another issue: local detail blurring in peripheral regions. Especially when the kernel size is scaled up to more than 50 or even 100 in the form of peripheral convolution, this phenomenon will be further amplified when a single parameter needs to process 8×8 or even 16×16 peripheral regions.

To solve, we propose the kernel-wise positional embedding. Formally, given a set of input features X, We process these features by a convolution with kernel weights $\mathrm{w} \in \mathbb{R}^{c_{\mathrm{in}} \times c_{\mathrm{out}} \times \mathrm{k} \times \mathrm{k}}$. We initialize the position embedding $\mathrm{h} \in \mathbb{R}^{c_{\mathrm{in}} \times \mathrm{k} \times \mathrm{k}}$ with *trunc normal* [49] initialization. The convolution process at the output position (x, y) can be represented as:

$$Y(x,y) = \sum_{i=-r_{\mathrm{w}}}^{r_{\mathrm{w}}} \sum_{j=-r_{\mathrm{w}}}^{r_{\mathrm{w}}} \mathrm{w}(i,j) \cdot \big(X(x+i, y+j) + \mathrm{h}(i,j)\big) \tag{3}$$

where Y is the output. $\mathrm{r_w}$ is the radius of the kernel w and we have $r_{\mathrm{w}} = \frac{\mathrm{k}-1}{2}$.

As illustrated in Fig. 3, by introducing kernel-wise positional embedding for kernel, we can distinguish specific locations in shared areas, so as to make up for the problem of vague local details caused by sharing. Actually, this can be viewed as adding bias with relative position information to the input features. It is worth noting that all the kernels in a stage share the same positional embedding h, thus the additional parameters brought by h are negligible. This design solves the position insensitivity problem caused by sharing weights in a cheap and elegant way, especially for extremely large kernels, e.g., 51×51 and 101×101.

4.3. Partial Peripheral Convolution

Large kernel convnets have been shown to have high channel redundancy [53] and suit well with sparsity [23]. Since our peripheral convolution enables us to design larger dense convolution with stronger spatial perception ability, we hope to further exploit the channel redundancy of large convolution. We introduce an Inception-style design where only partial channels of the feature map will be processed by convolution. We follow a simple philosophy: more identity mapping to exploit the channel redundancy. Specifically, for input X, we split it into two groups along the channel dimension,

$$\begin{aligned} X_{\text{conv}}, X_{\text{id}} &= \text{Split}(X) \\ &= X_{:,:,:g}, X_{:,:,g:} \end{aligned} \tag{4}$$

where g is the channel numbers of convolution branches and set to $\frac{3}{8}C_{in}$ by default. Then the split inputs are fed into peripheral convolution and identity mapping respectively,

$$\begin{aligned} X^{'}_{\text{conv}} &= \text{Peripheral Conv}(X_{\text{conv}}) \\ X^{'}_{\text{id}} &= X_{\text{id}} \end{aligned} \tag{5}$$

Finally, the outputs from two branches are concatenated to restore the original shape,

$$X^{'} = \text{Concat}(X^{'}_{\text{conv}}, X^{'}_{\text{id}}). \tag{6}$$

This design can be seen as a special case of Inception-style structure, such as Inception [37], Shufflenet [28, 55] and InceptionNeXt [53]. They utilize different operators in parallel branches while we take a much simpler philosophy: only peripheral convolution and identity mapping. We empirically find that this design suits well for peripheral convolutions with extremely large kernels, significantly reducing FLOPs without backfiring performance.

4.4. Architecture Specification

Built on the above designs and observations, we now elaborate the architectures of our Parameter-efficient Large Kernel Network (PeLK). We mainly follow ConvNeXt and SLaK to construct models with several sizes. Specifically, PeLK also adopts a 4-stage framework. We build the stem with a convolution layer with 4×4 kernels and 4 stride. The block numbers of stages are $[3, 3, 9, 3]$ for tiny size and $[3, 3, 27, 3]$ for small/base size. The kernel sizes for PeLK's different stages are $[51, 49, 47, 13]$ by default. For PeLK-101, the kernel sizes are scaled up to $[101, 69, 67, 13]$.

By default, we keep the central 5×5 region to be fine-grained. For PeLK-101, we enlarge the central region to 7×7 to adjust the increased kernel. Following SLaK, we also use dynamic sparsity to enhance model capacity. All the hyperparameters are set the same ($1.3\times$ width, 40% sparsity). We give thorough ablations for kernel configurations in section 5.4.

5. Experiments

In this section, we first conduct experiments on various essential vision tasks to evaluate PeLK with state-of-the-art baselines. Then in section 5.4 we comprehensively ablate on the design principles of our peripheral convolution.

5.1. Semantic Segmentation

For semantic segmentation, we evaluate PeLK backbones on the ADE20K benchmark [56], which consists of 25K images and 150 semantic categories. We use the UperNet [51] task layer for semantic segmentation. Following Swin and ConvNeXt, We train Upernet for 160K iterations with single-scale inference. The results are reported in Table 2 with mean Intersection of Union (mIoU) as the evaluation metric. Our proposed PeLK exceeds previous state-of-the-art models with remarkable improvements, demonstrating the effectiveness of our framework.

5.2. Object Detection

For object detection/segmentation, we conduct experiments with Cascade Mask R-CNN [3, 17] on MS-COCO [22]. Following ConvNeXt, we use the multi-scale setting and default configurations in MMDetection [4]. The Cascade Mask R-CNN model is trained with the 3x (36-epoch) training schedule. As shown in Table 3, PeLK achieves higher mAP than state-of-the-art methods, samely validating our superiority.

5.3. ImageNet Classification

The ImageNet-1K [8] dataset consists of 1000 object classes with 1.28M training images and 50,000 validation images. We extend the aforementioned training schedule in Section 3 to 300 epochs for a fair comparison. we

Table 2. Semantic segmentation comparison on ADE20K of different methods. We report the single-scale mIoU following ConvNeXt and SLaK. FLOPs are based on input sizes of (2048, 512).

Method	Kernel size	Params (M)	FLOPs (G)	mIoU (%)
Swin-T [24]	N/A	60	945	44.5
ConvNeXt-T [25]	7-7-7-7	60	939	46.0
SLaK-T [23]	51-49-47-13	64	957	47.6
PeLK-T	51-49-47-13	62	970	**48.1**
Swin-S [24]	N/A	81	1038	47.6
ConvNeXt-S [25]	7-7-7-7	82	1027	48.7
SLaK-S [23]	51-49-47-13	89	1057	49.4
PeLK-S	51-49-47-13	84	1077	**49.7**
Swin-B [24]	N/A	121	1188	48.1
ConvNeXt-B [25]	7-7-7-7	122	1170	49.1
RepLKNet-B [11]	31-29-27-13	112	1170	49.9
SLaK-B [23]	51-49-47-13	131	1210	50.2
PeLK-B	51-49-47-13	126	1237	**50.4**
PeLK-B-101	101-69-67-13	126	1339	**50.6**

Table 3. Object detection comparison on COCO of different methods. FLOPs are based on input sizes of (1280, 800).

Method	Params (M)	FLOPs (G)	AP^{box}	AP^{mask}
Swin-T [24]	86	745	50.5	43.7
ConvNeXt-T [25]	86	741	50.4	43.7
PeLK-T	86	770	**51.4**	**44.6**
Swin-S [24]	107	838	51.8	44.7
ConvNeXt-S [25]	108	827	51.9	45.0
PeLK-S	108	874	**52.2**	**45.3**
Swin-B [24]	145	982	51.9	45.0
RepLKNet-B [11]	137	965	52.2	45.2
SLaK-B [23]	152	1001	52.5	45.5
ConvNeXt-B [25]	146	964	52.7	45.6
PeLK-B	147	1028	**52.9**	**45.9**
PeLK-B-101	147	1127	**53.1**	**46.1**

conduct experiments for PeLK-T/S/B with input resolution 224×224. For PeLK-B and PeLK-B-101, we further experiment with input resolution of 384×384. More details of the training configurations can be found in Appendix A.

We compare PeLK with other state-of-the-art architectures under similar model size and FLOPs. As shown in Table 4, our model outperforms powerful modern CNNs and transformers like ConvNeXt [25] and Swin [24] by large margins. Notably, further scaling up the kernel size to extremely large (e.g., PeLK-101) can achieve consistent improvements. It is important to note that very large dense kernels are not intended for ImageNet classification, but our PeLK still exhibits a promising performance.

Table 4. Image classification accuracy (%) comparison on ImageNet-1K. We report the top-1 accuracy. Although very large dense kernels are not intended for ImageNet classification, our PeLK still exhibits a promising performance.

Method	Input size	Params (M)	FLOPs (G)	Top-1 acc
Swin-T [24]	224^2	28	4.5	81.3
T2T-ViT$_t$-14 [54]	224^2	22	6.1	81.7
PerViT-S [29]	224^2	21	4.4	82.1
ConvNeXt-T [25]	224^2	29	4.5	82.1
PeLK-T	224^2	29	5.6	**82.6**
PVT-Large [47]	224^2	61	9.8	81.7
T2T-ViT$_t$-19 [54]	224^2	39	9.8	82.4
PerViT-M [29]	224^2	44	9.0	82.9
Swin-S [24]	224^2	50	8.7	83.0
ConvNeXt-S [25]	224^2	50	8.7	83.1
PeLK-S	224^2	50	10.7	**83.9**
DeiT-B/16 [41]	224^2	87	17.6	81.8
RepLKNet-31B [11]	224^2	79	15.3	83.5
Swin-B [24]	224^2	88	15.4	83.5
ConvNeXt-B [25]	224^2	89	15.4	83.8
SLaK-B [23]	224^2	95	17.1	84.0
PeLK-B	224^2	89	18.3	**84.2**
ViT-B/16 [12]	384^2	87	55.5	77.9
DeiT-B/16 [41]	384^2	87	55.4	83.1
Swin-B [24]	384^2	88	47.1	84.5
RepLKNet-31B [11]	384^2	79	45.1	84.8
ConvNeXt-B [25]	384^2	89	45.0	85.1
SLaK-B [23]	384^2	95	50.3	85.5
PeLK-B	384^2	89	54.0	**85.6**
PeLK-B-101	384^2	90	68.3	**85.8**

5.4. Ablation Studies

Ablation on the sharing grid. We dive into what kind of sharing and granularity benefits most. For ease of understanding, we firstly give two instances to clearly indicate the sharing grid. For example, in Fig. 1a, we parameterize a 5×5 convolution using a 3×3 convolution, where the corresponding sharing grid is $[2, 1, 2]$. Each number represents the grid size parameterized by a single parameter. For Fig. 1b, we parameterize 31×31 convolution with a 11×11 convolution, the corresponding gird is $[7, 4, 2, 1, 1, 1, 1, 1, 2, 4, 7]$. Since the grid is symmetric at the center 1 (which is the central point in the kernel), we denote only half grid in Table 5 for simplicity.

We conduct experiments with the same 120-epoch schedule on ImageNet as in Section 3. We use PeLK-T without dynamic sparsity to give a sheer ablation on the sharing grid. For the baseline, we make the sharing grid to be all one (i.e., [1, 1, ..., 1]), in this way, it is equal to a 33×33 dense convolution as taken in RepLKNet. Results in Table 5 demonstrate that: **1)** the central fine granularity is of vital importance, while the peripheral regions can withstand

Table 5. Ablation study on sharing grid. No kernel-wise positional embedding is used.

#	Sharing Grid	Param	Top-1 Acc
1	[1, 1, ..., 1, 1]	1.00×	81.4
2	[2, 2, 2, 2, 2, 2, 2, 2, 1]	0.27×	81.0
3	[2, 2, 2, 2, 2, 2, 2, 1, 1, 1]	0.33×	81.4
4	[4, 4, 4, 2, 1, 1, 1]	0.16×	81.3
5	[8, 4, 2, 1, 1, 1]	0.11×	81.4
6	[1, 1, 2, 4, 8, 1]	0.11×	80.5

Table 6. Ablation on the central fine-grained kernel size. Kernel-wise positional embedding is used.

Sharing Grid	Central Kernel	Ratio	Top-1 Acc
[11, 8, 4, 2, 1]	1×1	0.04%	80.8
[10, 8, 4, 2, 1, 1]	3×3	0.35%	81.1
[9, 8, 4, 2, 1, 1, 1]	5×5	0.96%	81.6
[8, 8, 4, 2, 1, 1, 1, 1]	7×7	1.88%	81.6

wide range of sharing. # 2, 3 show that keeping the central 5×5 region unshared is the key to keep performance; # 3, 4, 5 exhibit that sharing in peripheral regions will not backfire performance evidently. We term this characteristic as focus-and-blur mechanism; **2)** an exponentially-increasing grid works best. Comparing # 4 with # 5, exponential gird not only reduces the parameters needed but also boosts the accuracy. From the above analysis, it can be seen that our design enjoys both the least amount of parameters and the highest performance.

Ablation on the central fine-grained area ratio. Table 6 ablates the effect of varying central fine-grained kernel size (i.e., the focus region). We also report the proportion of the central region to the total kernel size. The results show that the central region only takes about 1% proportion to maintain the model's high performance. However, the central region can not be too small, which will lead to severe performance degradation. Further increasing the central region does not bring additional benefits, but it brings additional parameters. In our main experiments, we keep the central 5×5 region of PeLK as fine-grained, and for PeLK-101, we enlarge the central region to 7×7 to maintain a similar central ratio.

Ablation on the kernel configuration. Table 7 ablates the configuration of kernel size in a 120 epoch schedule as in Section 3. For the input resolution of 224^2, enlarging kernel size to 101×101 will not bring additional benefits; while for input resolution of 384^2, PeLK-101 obtains a clear advantage over PeLK. Increasing kernel size to 152×152 leads to performance degradation, especially for input resolution of 224^2. These phenomena are reasonable considering the input resolution. For a typical convnet like ConvNeXt or our PeLK, the stem layer will result in a $4\times$ down-

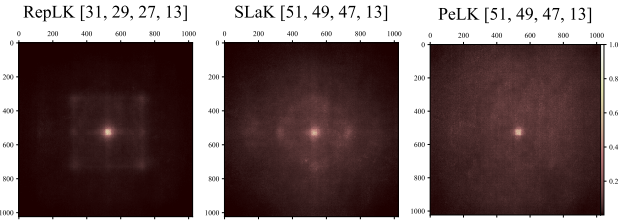


Figure 4. **Effective receptive field (ERF) comparison.** Our PeLK has larger ERFs than SLaK and RepLK, spreading a wider area.

sampling of the input images. So for input 224^2, a 51×51 kernel is roughly able to cover the global feature map after stem. And for input 384^2, a 101×101 kernel is equal to a global convolution, thus continuing scaling up kernel can not bring more global perception but only wasted parameters. This essentially suggests that kernel configuration should be tightly related to the input size. Currently, for the most commonly used 224^2 and 384^2 training, PeLK and PeLK-101 are the suitable options respectively. Moreover, with the development of hardware devices and computing power in the future, our approach will hopefully shine further when it is affordable to pretrain at higher resolutions.

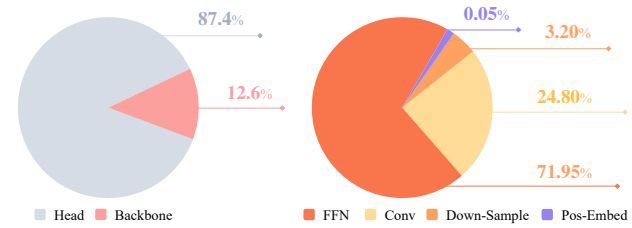
6. Analysis

6.1. Visualization of ERFs.

Previous large kernel convnets like RepLKNet and SLaK attribute their performance gains to their large Effective Receptive Fields (ERFs). Facilitated by peripheral convolution, PeLK has a much larger perception range. Therefore, we argue that PeLK's strong performance comes from larger ERFs. To verify, we depict the ERFs following RepLKNet and SLaK, we sample and resize 50 images from the validation set to 1024×1024, and measure the contribution of the pixel on input images to the central point of the feature map generated in the last layer. The contribution scores are further accumulated and projected to a 1024×1024 matrix, as visualized in Fig 4. Our PeLK spreads high-contribution pixels in a much larger ERF, validating our hypothesis and further exhibiting our effectivess.

Table 7. Ablation on the kernel size configuration. Kernel-wise positional embedding is used.

Model	Input Size	Kernel Size	Top-1 Acc
PeLK	224×224	51-49-47-13	81.6
PeLK-101	224×224	101-69-67-13	81.6
PeLK-151	224×224	151-89-87-13	81.2
PeLK	384×384	51-49-47-13	82.7
PeLK-101	384×384	101-69-67-13	83.0
PeLK-151	384×384	151-89-87-13	82.8



(a) FLOPs proportion of head & backbone (b) FLOPs proportion of backbone's components

Figure 5. **Analysis of FLOPs.** (a) FLOPs proportion of head & backbone. (b) FLOPs proportion of backbone's components. The head is UperNet and the backbone is PeLK-T respectively. FLOPs are based on input sizes of (2048, 512).

6.2. Analysis of FLOPs

We provide a detailed breakdown of the FLOPs for the PeLK-T architecture utilized in semantic segmentation in Fig.5. As shown in Fig.5(a), we depict the FLOPs distribution between the head (i.e., UperNet [51]) and backbone (i.e., PeLK-T) of the model. In Fig.5(b), we give a comprehensive analysis of the FLOPs contributions from different components of the backbone (i.e., PeLK-T), including FFNs, large-kernel convolutions, down-sampling layers, and kernel-wise positional embedding. There are two noteworthy points. Firstly, large kernel convolutions account for approximately 25% of the overall FLOPs of the backbone, thus further scaling up the kernel size does not significantly increase the overall FLOPs. Secondly, the extra FLOPs introduced by positional embedding are minimal, accounting for only 0.05% of the backbone's FLOPs. So, kernel-wise positional embed is both cheap and elegant.

6.3. Inference Throughput Measurement

We compare inference throughput measurement in Table 8. The results are obtained on an A100 GPU with input resolution of 224×224. We use PyTorch 1.10.0 + cuDNN 8.2.0 and FP32 precision. Although SLaK uses stripe convolution to speed up the computation of very large kernel, we still hold a clear speed advantage (i.e., $1.5\times$ speedup). This advantage is particularly remarkable considering that PeLK outperforms SLaK on ADE20K, COCO and ImageNet. More importantly, scaling up kernel to 101 only brings minor speed overhead, further exhibiting our design's merits in scaling properties.

Table 8. Inference throughput comparison on ImageNet-1K. The results are in FP32 precision. We use an A100 GPU with PyTorch 1.10.0 + cuDNN 8.2.0 to conduct this experiment.

Models	Input	Kernel Size	Throughput
SLaK-T [23]	224^2	51-49-47-13	754
PeLK-T	224^2	51-49-47-13	**1138**
PeLK-101-T	224^2	101-69-67-13	**1077**

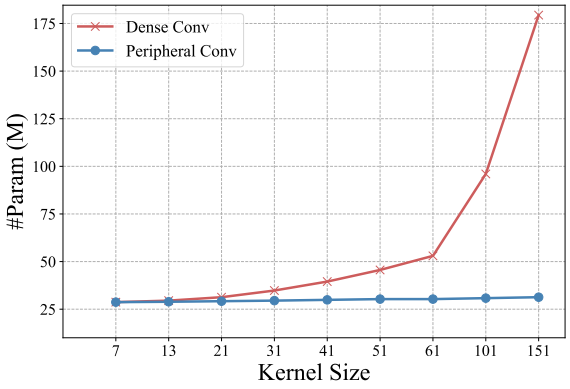


Figure 6. **Scaling efficiency comparison.** We compare the model size with a set of kernel sizes from 7 to 151. Our peripheral convolution has a clear advantage, bringing minor parameter overhead.

6.4. Kernel Scaling Efficiency.

Our peripheral convolution reduces the parameter complexity of dense convolutions from $O(K^2)$ to $O(\log K)$, which enables us to scale up kernel size with a remarkably minor model size overhead. To demonstrate this, we simply replace all the kernels in stages of ConvNeXt-T with a set of kernel sizes from 7 to 151 and report the required number of parameters. As shown in Fig 6, our approach exhibits a remarkable scaling advantage, and we can see a clear gap when the kernel size is larger than 50. Using dense convolution results in a rapidly growing model size, which is unacceptable in practice. In contrast, our peripheral convolution incurs only a minor model size overhead, making it possible to design extremely large kernel convnets.

7. Conclusion

This paper explores the design of extremely large kernel convolutional neural networks. We propose a new form of convolution termed peripheral convolution, which can reduce the parameter complexity of dense convolution from $O(K^2)$ to $O(\log K)$ while keeping dense convolution's merits. Built upon the proposed peripheral convolution, we design extremely large dense kernel CNNs and achieve notable improvements across a variety of vision tasks. Our strong results suggest biologically inspired mechanisms can make a promising tool to boost modern network design.

Acknowledgments

This work is supported in part by the National Key R&D Program of China (Grant No.2022ZD0116403), the National Natural Science Foundation of China (Grant No. 61721004), and the Strategic Priority Research Program of Chinese Academy of Sciences (Grant No. XDA27000000).

We thank Yurong Zhang for the help in the depiction of Fig.1(b) and Bo Zhang for technical support.

References

[1] Megengine:a fast, scalable and easy-to-use deep learning framework. https://github.com/MegEngine/MegEngine, 2020. 3

[2] Benjamin Balas, Lisa Nakano, and Ruth Rosenholtz. A summary-statistic representation in peripheral vision explains visual crowding. *Journal of vision*, 9(12):13–13, 2009. 2, 3

[3] Zhaowei Cai and Nuno Vasconcelos. Cascade r-cnn: Delving into high quality object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6154–6162, 2018. 5

[4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, et al. Mmdetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019. 5

[5] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. Largekernel3d: Scaling up kernels in 3d sparse cnns. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13488–13498, 2023. 2

[6] Xiangxiang Chu, Zhi Tian, Yuqing Wang, Bo Zhang, Haibing Ren, Xiaolin Wei, Huaxia Xia, and Chunhua Shen. Twins: Revisiting the design of spatial attention in vision transformers. *Advances in Neural Information Processing Systems*, 34:9355–9366, 2021. 1

[7] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020. 3

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 5

[9] Arturo Deza and Miguel Eckstein. Can peripheral representations improve clutter metrics on complex scenes? *Advances in neural information processing systems*, 29, 2016. 2, 3

[10] Arturo Deza and Talia Konkle. Emergent properties of foveated perceptual systems. *arXiv preprint arXiv:2006.07991*, 2020. 2, 3

[11] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in cnns. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11963–11975, 2022. 1, 2, 3, 6

[12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1, 2, 6

[13] Lex Fridman, Benedikt Jenik, Shaiyan Keshvari, Bryan Reimer, Christoph Zetzsche, and Ruth Rosenholtz. Sideeye: A generative neural network based simulator of human peripheral vision. *arXiv preprint arXiv:1706.04568*, 2017. 3

[14] Stephen Gould, Joakin Arfvidsson, Adrian Kaehler, Benjamin Sapp, Marius Messner, Gary Bradski, Paul Baumstarck, Sukwon Chung, Andrew Y Ng, et al. Peripheral-foveal vision for real-time object recognition and tracking in video. 2007.

[15] Anne Harrington and Arturo Deza. Finding biological plausibility for adversarially robust features via metameric tasks. In *SVRHM 2021 Workshop@ NeurIPS*, 2021. 3

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1

[17] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 5

[18] Aditya Jonnalagadda, William Yang Wang, BS Manjunath, and Miguel P Eckstein. Foveater: Foveated transformer for image classification. *arXiv preprint arXiv:2105.14173*, 2021. 3

[19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. 1, 2

[20] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 1

[21] Jerome Y Lettvin et al. On seeing sidelong. *The Sciences*, 16(4):10–20, 1976. 2

[22] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014. 5

[23] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Mykola Pechenizkiy, Decebal Mocanu, and Zhangyang Wang. More convnets in the 2020s: Scaling up kernels beyond 51x51 using sparsity. *arXiv preprint arXiv:2207.03620*, 2022. 1, 2, 3, 5, 6, 8

[24] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 1, 2, 6

[25] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022. 1, 2, 6

[26] Chin Ian Lou, Daria Migotina, Joao P Rodrigues, Joao Semedo, Feng Wan, Peng Un Mak, Pui In Mak, Mang I Vai, Fernando Melicio, J Gomes Pereira, et al. Object recognition test in peripheral vision: a study on the influence of object color, pattern and shape. In *Brain Informatics: International Conference, BI 2012, Macau, China, December 4-7, 2012. Proceedings*, pages 18–26. Springer, 2012. 2, 3

[27] Hristofor Lukanov, Peter König, and Gordon Pipa. Biologically inspired deep learning model for efficient foveal-peripheral vision. *Frontiers in Computational Neuroscience*, 15:746204, 2021. 3

[28] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*, pages 116–131, 2018. 5

[29] Juhong Min, Yucheng Zhao, Chong Luo, and Minsu Cho. Peripheral vision transformer. *Advances in Neural Information Processing Systems*, 35:32097–32111, 2022. 3, 6

[30] Chao Peng, Xiangyu Zhang, Gang Yu, Guiming Luo, and Jian Sun. Large kernel matters–improve semantic segmentation by global convolutional network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4353–4361, 2017. 2

[31] RT Pramod, Harish Katti, and SP Arun. Human peripheral blur is optimal for object recognition. *Vision research*, 200: 108083, 2022. 4

[32] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in Neural Information Processing Systems*, 34:12116–12128, 2021. 1

[33] Ruth Rosenholtz. Capabilities and limitations of peripheral vision. *Annual review of vision science*, 2:437–457, 2016. 2

[34] Ruth Rosenholtz. Demystifying visual awareness: Peripheral encoding plus limited decision complexity resolve the paradox of rich visual experience and curious perceptual failures. *Attention, Perception, & Psychophysics*, 82(3):901–925, 2020. 2, 3

[35] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 1, 2

[36] Hans Strasburger, Ingo Rentschler, and Martin Jüttner. Peripheral vision and pattern recognition: A review. *Journal of vision*, 11(5):13–13, 2011. 2

[37] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015. 5

[38] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015. 2

[39] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016. 2

[40] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems*, 34:24261–24272, 2021. 2

[41] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 6

[42] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 1

[43] Asher Trockman and J Zico Kolter. Patches are all you need? *arXiv preprint arXiv:2201.09792*, 2022. 2

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1

[45] Manish Reddy Vuyyuru, Andrzej Banburski, Nishka Pant, and Tomaso Poggio. Biologically inspired mechanisms for adversarial robustness. *Advances in Neural Information Processing Systems*, 33:2135–2146, 2020. 3

[46] Panqu Wang and Garrison W Cottrell. Central and peripheral vision for scene recognition: A neurocomputational modeling exploration. *Journal of vision*, 17(4):9–9, 2017. 3

[47] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 568–578, 2021. 1, 6

[48] William H Warren and Kenneth J Kurtz. The role of central and peripheral vision in perceiving the direction of self-motion. *Perception & psychophysics*, 51(5):443–454, 1992. 2

[49] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019. 4

[50] Maarten WA Wijntjes and Ruth Rosenholtz. Context mitigates crowding: Peripheral object recognition in real-world images. *Cognition*, 180:158–164, 2018. 2, 3

[51] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European conference on computer vision (ECCV)*, pages 418–434, 2018. 5, 8

[52] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European conference on computer vision (ECCV)*, pages 418–434, 2018. 3

[53] Weihao Yu, Pan Zhou, Shuicheng Yan, and Xinchao Wang. Inceptionnext: When inception meets convnext. *arXiv preprint arXiv:2303.16900*, 2023. 5

[54] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from

scratch on imagenet. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 558–567, 2021. 6

[55] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6848–6856, 2018. 5

[56] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ade20k dataset. *International Journal of Computer Vision*, 127:302–321, 2019. 5